PDO

Base de datos

En un documento php dentro de la carptea de lib que luego debemos linkear en cada uno de los otros doc con un `Include_once("libs/data_base.php");`

Conexión

Para abrir la conexión a través del método PDO es necesario establecer unas variables antes. En ellas se establecerá el nombre del servidor,el usuario y contraseña y el nombre de la base de datos.

```
    $servername='db';
    $username='root';
    $password='test';
    $dbname='donacion';
```

Una vez que esto se ha decidido, abrimos un try/catch para establecer la conexión y recoger el error en el caso de que suceda.

```
    try{
        $connection = new PDO("mysql:host=$servername;dbname=$dbname",
$username,$password);
        $connection ->
setAttribute(PDO::ATTR_ERRMODE,PDO::ERRMODE_EXCEPTION);
        echo "Database connected successfully";
        return $connection;
    } catch (PDOException $e){
        echo "Connection failed: " . $e->getMessage();
    }
```

- `$connection` Es la variable que usaremos para poder trabajar en el resto de la función.
- `new PDO` Es el indicador de que se abre una nueva conexión PDO.
- `mysql` Indica el sistema de gestión de datos que se utilizará. A la que se alsiganarán las variables de antes para completar la información.

Despues de esto le decimos como manejar errores:

```
$connection -> setAttribute(PDO::ATTR_ERRMODE,PDO::ERRMODE_EXCEPTION);
```

- `setAttribute` Es el método por el cual podemos modificar los atributos de la concexión.
- `PDO::ATTR_ERRMODE` Especifica el modo en que PDO manejará los errores.
- `PDO::ERRMODE_EXCEPTION` Indica que PDO debe lanzar una excepción si ocurre un error.

```
return $connection
```

Lo que devuelve la conexión en caso de ser positiva para proceder con la petición.

Bloque catch

Se recupera la excepción con PDOException y se guarda en la variable `$e` y con echo imprimimos el mensaje a través de `getMessage()`.

Crear DB

```
function create_DB ($connection){
    $sql="CREATE DATABASE IF NOT EXISTS DONACION";
    executeQuery($connection,$sql);
}
```

Creamos la petición y la guardamos en una variable. Por último con la función creada por nosotros, le pasamos la concexion y la peticion se ejecuta. Esta estructura será ña utilizada en cualquiera de las otras peticiones a la DB.

Ejecucion de petición

```
function executeQuery($connection,$sql){
    try{
        $connection->query($sql);
    }catch(PDOException $e){
        echo $e->getMessage();
    }
}
```

Debemos pasarle la variable con la conexión y petición y con `$connection->query($sql);` se ejecuta la peticion. Esto dentro de una estructura try/catch como la que se hizo antes.

Procesado en las páginas

La estructura que a de seguir en cada página en la que debamos interactuar con un formulario se divide la página en 2 secciones para el php.

Creacion en el server

En la página principal se debe crear la DB y las tablas, al menos la primera vez. Por lo que es omportante en las funciones encargadas de estas creaciones, poner un IF NOT EXISTS

```
$connection=getConnection();
create_DB($connection);
select_DB($connection);
create_table_admin($connection);
create_table_donantes($connection);
create_table_historico($connection);
```

Retroalimentación

En cada formulario que tengamos que realizar se validar los datos antes de que se envien. Para ello, las primeras lineas en el body deben hacer referencia a esto:

```
<?php include_once ("style/header.html");
$mensaje="";
if($_SERVER['REQUEST_METHOD']=="POST" && isset($_POST['submit'])){
    $nombre= validate($_POST['name']);
    $apellidos=validate($_POST['apellidos']);
    $age=validate($_POST['age']);
    $grp_sangre=validate($_POST['grp_sangre']);
    $cod_postal=validate($_POST['cod_postal']);
    $tlf=validate($_POST['tlf']);

    $connection = getConnection();
    select_DB($connection);
}
```

- En el caso de querer incluir cualquier efecto estético como puede ser `include_once ("style/header.html");` debe realizarse antes también.
- Debemos checkear cuando carguemos la página si hay algo cargado con un if, lo normal es que en este momento este vacio. En el if filtramos por el tipo de formulario (get o post) y que se haya enviado.

```
if($_SERVER['REQUEST_METHOD']=="POST" && isset($_POST['submit']))
```

Para el primer paso(el tipo de formulario) usamos la variable `$_SERVER` para acceder al método de envio. Y con `isset($_POST)` sabemos si se ha activado el botón de submit.

En el caso de entrar en el if procedemos a validar todos los campos que queramos. Para ello podemos crear otro documento con todas las validaciones (también en libs), en este caso debemos incluir ese documento (antes de cualquier código). Cada campo validado se guarda en una variable que se reconzca para poder pasar las a la DB.

```
$conexion = get_conexion();
select_DB($conexion);
alta_usuario($conexion, $nombre, $apellidos, $edad, $provincia);
$conexion->close();
```

Primero abrimos conexión y la guardamos en la variable. Acto seguido llamamos a la funcion para conectarnos a la DB con esa conexión. Por último llamamos a la funcion que realice la peticion a la DB que busquemos, pasandole la conexion y los datos validados. Siempre cerrar la conexión a la base de datos, se puede hacer a través de una función o en cada documento.

Una vez tengamos esta parte lista podremos empezar con el formulario. La parte de PHP de los formularios se centra en la primera línea, en la pripia etiqueta del Form.

```
<Form method="post" action="<?php echo
htmlspecialchars($_SERVER['PHP_SELF']);?>">
```

En ella se indica el metodo por el que se va a enviar el formulario. En la segunda parte se referencia el donde se va a enviar. junto con `htmlspeciachars` para evitar problemas con código malicioso. y con la supervariable `$_SERVER`.

POO

Base de datos

En un documento php dentro de la carptea de lib que luego debemos linkear en cada uno de los otros doc con un `Include_once("libs/data_base.php");`

Conexión

```
function get_conexion(){
    $conexion = new mysqli('db','root','test');
    if($conexion->connect_errno != null){
        die("Fallo en la conexion: ".$conexion->connect_error." con número
".$conexion->connect_errno);
    }
    return $conexion;
}
```

En la primera línea le indicamos el sistema que se usara `new mysql` y le pasamos la información de la DB (nombre, usuario y contraseña) `('db','root','test')`.

En el caso de que devuleva desde la variable un error, debemos recogerlo y mostrarlo. Con un if, cuya condición sea que el error sea diferente a nulo, hacemos que imprima el mensage de error y que termine el evento usando `die`.

```
($conexion->connect_errno != null)
```

Crear DB

Para crear la DB debemos crear una función que pueda trabajar con las consultas a mysql. La estructura de esta función será usada para crear tablas, y otras consultas, solo cambiarán las variables que necesitan y la estructura interna.

```
function crear_DB($conexion){
    $sql = "Create database if not exists tienda";
    ejecutar_consulta($conexion, $sql);
}
```

Para crear esta función debemos facilitarle la conexión que hemos creada anteriormente. Una vez dentro crearemos una variable con la consulta deseada y llamaremos a nuestra función para ejecutarla.

Función ejecutar consultas

```
function ejecutar_consulta($conexion, $sql){
    $resultado = $conexion->query($sql);
    if ($resultado == false){
        die($conexion->error);
    }
    return $resultado;
}
```

Es necesario pasar la consulta y la conexión. Con la conexión ejecutamos el método query, pasandole la consulta. Y para manejar los errores con un of nos aseguramos que el evento es igual a false debemos matar el proceso e imprimir el error.

Procesado en las páginas

Creacion en el server

```
$conexion=get_conexion();
crear_DB($conexion);
select_DB($conexion);
create_user_table($conexion);
```

Parq iniciar la DB en el servidor debemos idicarselo en la primera página que se cargue. Es importante que en las funciones en las que la consulta implica la creación de estructuras, debemos asegurarnos de que especificamos el `IF NOT EXISTS` para que no creemos cada vez que iniciamos o nos alerte de error.

Retroalimentación

Igual que PDO.

Editar DB

En el caso de que queramos editar un elemtneo de nuestra base de datos, el usuario debe ver cargados los datos que ya tendríaos guardados para poder editarlos. En ese caso se añade una capa más de complejidad.

Para prepar la página inicial debemos generar una estructura php antes del código html. Debemos indicar y guardar en una variable la conexión a la DB, asi como asignar variables vacias a todos los elements que debe mostrar y editar el usuario.

Con una estructura if/else comprobamos si el usuario a mandado la informacion del registro. En el momento que se inicia esa pagina por primera vez esto va a ser negativo asi que saltará al else. Pero una vez editado si que entrará en esta parte que es exactamente la misma que la de añadir una persona (guardar en variables los valores validados) y se guardan con nuestra función de editar_user.

¿Pero que pasa la primera vez que se carga la página? Al entrar al else se encuentra con este código:

```
if(isset($_GET["id"])){
    $id_user=$_GET["id"];
    $user = buscar_usuario($conexion, $id_user);
    if($user->num_rows >0){
        $row=$user->fetch_assoc();
        $id_user=$row['id'];
        $nombre=$row['nombre'];
        $apellidos=$row['apellidos'];
        $edad=$row['edad'];
        $provincia=$row['provincia'];
    }
}else{
    $id_user=0;
    $nombre="";
    $apellidos="";
    $edad=0;
    $provincia="corunha";
}
```

En esta sección el if interior se asegura de recibir (por GET), en caso positivo busca con nuestra función el usuario y lo asigna a una variable. Con fetch dividimos por filas y reasignamos las variables del principio.

Por último, en el propio formulario en el apartado valor le asignamos el homónimo.

SEGUNDo TRIMESTRE

Sesiones `$_SESSION`

Iniciar sesión

Para iniciar una sesión se requiere la sentencia `session_start();`. En el caso de que existan otras sesiones pero no se les pase, se creará una nueva sesión, si no hay ninguna sesión se accede a la supervariable `$_SESSION`[^1]. La sintaxis para crear una sesión similar a una variable normal: `$_SESSION["favcolor"] = "verde";`. Donde es necesario indicar que es una sesión y enter parentesis el nombre que se le asigna a dicha sesión:

```
<?php
  // Start the session
  session_start();
?>
<!DOCTYPE html>
<html>
  <body>
  <?php
  // Establecer variables de sesión
  $_SESSION["favcolor"] = "verde";
  $_SESSION["favanimal"] = "gato";
  echo "Variables de sesión establecidas.";
  ?>
  </body>
</html>
```

[^1]: La mayoría de las sesiones configuran una clave de usuario en el navegador del usuario (similar a 765487cf34ert8dede5a562e4f3a7e12). Luego, cuando se abre una sesión en otra página, escanea en busca de una clave de usuario. Si hay una coincidencia, accede a esa sesión, si no, inicia una nueva sesión.

Obtener sesiones

A la hora de acceder a la sesión debemos referir al nombre, pero siempre indicando `$_SESSION`:

```
<?php
  session_start();
?>
<!DOCTYPE html>
<html>
  <body>
  <?php
  // Echo session variables that were set on previous page
  echo "El color favorito es: " . $_SESSION["favcolor"] . ".<br>";
  echo "El animal favorito es:  " . $_SESSION["favanimal"] . ".";
  ?>
  </body>
</html>
```

Todas las sesiones se almacenan en la variable global `$_SESSION`, por lo que una forma de acceder a todas las variables almacenadas sería:

```
<?php
  print_r($_SESSION);
?>
```

Se puede especificar que al obtener la información escojamos el id de la sesión: `echo 'A sesión actual é: '.session_id();`

Modificar/eliminar sesiones

Para modificar una sesión solo hay que sobreescribirla. En el caso de querer eliminarla la sintaxis es simple `unset($_SESSION["favcolor"]);`

Evitar ataques Session Fixation

Estos ataques se basan en los cuales es atacante regista una id en el server que luego se la pasa al usuario. Con esto en futuras ocasiones puede utilizar ese id para entrar como si fuera el usuario. Para evitar este tipo de ataques existen tres opciones:

Uso de cookies

En primer lugar podemos usar las cookies para evitar que la id viaje en la URL o en formularios. Para ello guardamos la session en alguna cookie y tendremos que recuperarla, esta situación reduce las posibilidades, pero nunca son cero. Con el siguiente código se hace obligatorio que la id de session solo se use si proviene de una cookie:

```
<?php
  ini_set('session.use_only_cookies',1);
?>
```

Marca de session

En este caso, una vez que se crea la session, se pregunta si ya fue creada o es nueva. En el caso de que la respues sea no, significa que es nueva o inyectada, por lo que entraría en el if. Dentro del if lo que sucede es:

- `session_regenerate_id(true);` - En este caso se crea una session nueva con diferente id, y se le pasa la información de la anterior. Buscando que el atacante ya no tenga el mismo id, pero el ususario no note diferencia en su session. Por último, se elimina la session anterior que podía estar coprometida.
- `$_SESSION['mimarcadecontrol'] = true;` - Una vez creada se le añade una marca interna para que en futuras comprobaciones el sistema sepa que fue creada por nosotros de forma legal.

```
session_start();

if (!isset($_SESSION['mimarcadecontrol'])){
    session_regenerate_id(true);
```

```
    $_SESSION['mimarcadecontrol'] = true;
}
```

Renovar al loggear

Al loguearse el usuario la session pasa de ser default a tener ciertos privilegios, es ese momento donde debemos cambiar el id de la seesion. Para eso usamos el mismo método que en el caso anterior, pero sin añadir una marca después (no lo vamos a comprobar luego). Otras ocasiones donde sería obligatorio este cambio de id son cambio de rol, activación de permisos especiales y cambio de contraseña.

```
if ($usuario_logueado === true){
    session_regenerate_id(true);
    $_SESSION['logueado'] = true;
}
```

Cookies

Crear cookies

La sintaxis de una cookie es simple, y de los valores que pide el único que es obligatorio es el nombre, es resto son opcionales. **Para settear una cookie se usan paréntesis**. `setcookie(name, value, expire, path, domain, secure, httponly);`

Recuperar cookies

Para poder recuperar una cookie debemos usar la variable global `$_COOKIE` . El siguiente código muestra como recuperar una cookie, junto con el paso previo de comprobar si existe esa cookie. **Para recuperar la cookie se utiliza los corchetes para encapsular los nombres de las variables**.

```
<?php
  $cookie_name = "usuario";
  $cookie_value = "Sabela";
  setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); //
86400 = 1 day
?>
<html>
  <body>
    <?php
    if(!isset($_COOKIE[$cookie_name])) {
        echo "La cookie con nombre '" . $cookie_name . "' no está
definida!";
    } else {
        echo "La cookie '" . $cookie_name . "' está definida!<br>";
        echo "Su valor es: " . $_COOKIE[$cookie_name];
    }
    ?>
  </body>
</html>
```

Modificar y eliminar

Al igual que en la session para modificar una cookie solo es necesario reescribirla. En el caso de querer eliminar una cookie se modifica su fecha de caducidad y se autoelimina.

Ficheros

Abrir/leer

Para poder abrir un fichero tenemos dos opciones: con "echo" se reproduce el texto de manera muy simple, sin respetar saltos de página ni formatos. Con la función `fopen()` se permiten más opciones. Es una forma de crear una lista de parámetros que se apliquen a la hora de abrir el documento. Primero se guarda en una variable en la que le indicamos que archivo sobre el que queremos que se apliquen, y los parámetros (en este caso se añade un "die" en caso de que no sea posible no genere errores). Después se hace un "echo" junto con la función "fread()" (que aplica los parámetros anteriores). Dentro de esta, esta la variable con los parámetros y el archivo. Por último, se cierra con la función "fclose()" una vez que se acaba de trabajar con el.

```
<?php
  $mifichero = fopen("webdictionary.txt", "r") or die("Unable to open
file!");
  echo fread($mifichero,filesize("webdictionary.txt"));
  fclose($mifichero);
?>
```

Los parámetros que se pueden añadir son:

- r/r+ - Solo de lectura o lectura/escritura con el cursor al principio.
- w/w+ - Solo escritura o lectura/escritura con el cursor al principio, y si no existe el fichero se crea. Que el cursor estea al principio implica que si se escribe algo se sobreescribe el contenido.
- a/a+ - Solo escritura o lectura/escritura con el cursor al final, y si no existe el fichero se crea. Que el cursor esté al final sigifica que al escribir algo, esto se añade a lo ya escrito.

Otras funciones

- fgets() - Sirve para leer solo una línea del documento.
- fgetc() - Sirve para leer un solo carácter.
- feof () - Sirve para comprobar si se llegó al final de documento.

```
<?php
  $mifichero = fopen("webdictionary.txt", "r") or die("Unable to open
file!");
  // Output one line until end-of-file
  while(!feof($mifichero)) {
    echo fgets($mifichero) . "<br>";
  }
```

```
    fclose($mifichero);
?>
```

Crear

Para crear también se utiliza la función `fopen()` porque se asume que se abre para escribir o agregar (`$mifichero = fopen("testfile.txt", "w")`).

```
<?php
    $mifichero = fopen("nuevoarchivo.txt", "w") or die("Unable to open file!");
    $txt = "Miguel\n";
    fwrite($mifichero, $txt);
    $txt = "Juan\n";
    fwrite($mifichero, $txt);
    fclose($mifichero);
?>
```

En este código se abre un cocumento (al no existir se crea), con el prametro de escritura y cursor al principio. Seguido de la inclusión de dos nombres. Si se repitiese este código con otros nombres se substituiría lo ya existente, sin embargo cambiando la "w" por "a" se añadirían.

Subir archivos

Para poder subir un archivo es necesario cumplir una serie de requisitos: que el formulario se envíe por método "post", debe tener el atributo `enctype="multipart/form-data"` y el input con el atributo `type = "file"` debe tener asociado un boton de "examinar" (normalmente los hace automaticamente el navegador).

Carga

- En la primera línea guardamos en una variable la ruta hasta la carpeta donde guardamos los archivos. Es necesario que exista previamente y que tenga permisos de escritura para que PHP pueda acceder.
- En la segunda línea guardamos en una variable el archivo. Primero concatenamos la ruta antes guardada con basename (toma solo el nombre del archivo, no la ruta para evitar malas intenciones). Por último tenemos $_FILES (como un array con los documentos que envió el usuario) donde se especifica el id del input ("fileToUpload") y luego el nombre del archivo.
- Por último, para verificar el tipo de archivo se obtine el extensión `pathinfo($target_file, PATHINFO_EXTENSION)`. Para evitar problemas en la comprobación se convierten a minúsculas. Esto se guarda en una variable para poder realizar esta comparación. o limitaciones por tipos

```
    //Carpeta donde se van a incluir los ficheros
    $target_dir = "uploads/";
    //Recuperamos el nombre del fichero
    $target_file = $target_dir . basename($_FILES["fileToUpload"]["name"]);
```

```
    //Obtenemos el tipo del fichero
    $imageFileType = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
```

Para comprobar si el archivo existe podemos tomar la variable del nombre del archivo.

```
if (file_exists($target_file)) {
    die("El fichero ya existe.");
}
```

En caso de querer confirmar o limitar por tamaño podemos usar parte del código que interviene en la recuperacon del nombre:

```
if ($_FILES["fileToUpload"]["size"] > 500000) {
    die("El archivo es demasiado grande."); // Detener el script
}
```

PHP Debug

Para poder usar el Xdebug debemos indicarlo en el Dockerfile

```
#Xdebug
RUN pecl install xdebug-3.3.2
ADD xdebug.ini /usr/local/etc/php/conf.d/xdebug.ini
```

Además de tener un archivo xdebug.ini para establecer unas variables de entorno:

```
# docker/php/xdebug.ini
zend_extension=xdebug
[xdebug]
xdebug.mode=develop,debug
xdebug.start_with_request=yes
xdebug.client_host='host.docker.internal'
```

A mayores tener el archivo launch.json en la carpeta .vscode paraque pueda interpretar de manera correcta las instrucciones del xdebug.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Listen for Xdebug",
            "type": "php",
```

```
            "request": "launch",
            "port": 9003,
            "pathMappings": {
                "/var/www/html": "${workspaceFolder}/www"
              }
        },
        {
            "name": "Launch currently open script",
            "type": "php",
            "request": "launch",
            "program": "${file}",
            "cwd": "${fileDirname}",
            "port": 0,
            "runtimeArgs": [
                "-dxdebug.start_with_request=yes"
            ],
            "env": {
                "XDEBUG_MODE": "debug,develop",
                "XDEBUG_CONFIG": "client_port=${port}"
            }
        },
        {
            "name": "Launch Built-in web server",
            "type": "php",
            "request": "launch",
            "runtimeArgs": [
                "-dxdebug.mode=debug",
                "-dxdebug.start_with_request=yes",
                "-S",
                "localhost:0"
            ],
            "program": "",
            "cwd": "${workspaceRoot}",
            "port": 9003,
            "serverReadyAction": {
                "pattern": "Development Server \\(http://localhost:([0-
9]+)\\) started",
                "uriFormat": "http://localhost:%s",
                "action": "openExternally"
            }
        }
    ]
}
```

Filtrado de datos

Existen varias formas de filtrado de datos: validacion (EJ para email: `FILTER_VALIDATE_EMAIL`) o saneamiento (Ej para email: `FILTER_SANITIZE_EMAIL`). En el primer caso, si la información proporcionada no cumple con lo que se pide no se tomará, en el segundo se eliminan aquellos elementos que no se permitan. en el caso de los arrays se aplica una sintaxis mas compleja:

```
filter_var_array(array $data, array|int $args, bool $add_empty = true):
array|false|null
```

Un ejemplo con todos los filtros aplicaldos sería:

```
$data = [
    'nombre' => 'Iván Gómez',
    'email' => 'ivan@example.com',
    'edad' => '18'
];
$args = [
    'nombre' => FILTER_SANITIZE_STRING, // Elimina caracteres dañinos de la
cadena
    'email' => FILTER_VALIDATE_EMAIL,  // Valida que el email sea válido
    'edad' => [
        'filter' => FILTER_VALIDATE_INT, // Valida que sea un entero
        'options' => ['min_range' => 18, 'max_range' => 65], // Rango
permitido
    ]
];
$result = filter_var_array($data, $args);
print_r($result);
```

Variables de entorno

Este tipo de variables afectan a dos ficheros docker-compose.yml y .env, y buscan guardar cierto tipo de informacion para que sea igual en todas las construcciones. En el docker-compose.yml las variables que se agrupan dentro de "enviroment" se transferirán a .env, donde no precisa ninguna estructura especfífica (un simple corta y pega).

```
services:
    www:
        build: .
        ports:
            - "80:80"
        volumes:
            - ./www:/var/www/html
        links:
            - db
        networks:
            - default
        extra_hosts:
            - "host.docker.internal:host-gateway"
        environment:
            APP_ENV: development
            APP_DEBUG: "true"
            DATABASE_HOST: db
```

```
            DATABASE_USER: colegio
            DATABASE_PASSWORD: colegio
```

Siendo sustituido por:

```
services:
    www:
        build: .
        ports:
            - "80:80"
        volumes:
            - ./www:/var/www/html
        env_file:
            - .env
```

Login y sesiones

Para poder tener cierto control sobre los ususrios regostrados usaremos un pequeño formulario para luego comprar con los usuarios que tengamos registrados.

```
<?php
session_start();
function comporbar_usuario($nombre, $pass)
{
  if($nombre == "usuario" && $pass="abc123."){
      $usuario['nombre']="usuario";
      $usuario['rol']=0;
      return $usuario;
  }elseif($nombre == "admin" && $pass="1234"){
      $usuario['nombre']="admin";
      $usuario['rol']=1;
      return $usuario;
  }else{
      return false;
  }
}
```

```
//Comprobar si se reciben los datos
if($_SERVER["REQUEST_METHOD"]=="POST"){
   $usuario = $_POST["usuario"];
   $pass = $_POST["pass"];
   $user = comporbar_usuario($usuario, $pass );
   if(!$user){
      $error = true;
   }else{
      $_SESSION['usuario'] = $user;
      //Redirigimos a index.php
```

```
        header('Location: index.php');
    }
}
```

En el caso de querer eliminar la sesión con el fin de cerrarla o cambiarla debemos destruirla

```
<?php
    session_start();
    $_SESSION = array();
    session_destroy();
    header("Location: index.php");
?>
```

Clases y objetos

Clase y objeto

Los objetos son instancias de una clase, y las clases son estructuras base para la creación de objetos. Esta estructura básica tiene una serie de puntos obligatorios: Para empezar debe declararse como `class` y debe ser nombrada en mayuscula. Lo primero que encontramos en la clase son las variables que formarán el objeto (en este casonombre y color), para que no se pueda acceder a estas variables que forman la estructura basica de los ibjetos debemos usar los setters y getters. Estos permiten variar el valor de las variables pero no cambiar las propias variables, es decir, podemos variar el valor de la variable color, pero no eliminar la variable color de la estructura.

```
class Fruit {
  // Propiedades
  public $name;
  public $color;
  // Métodos
  function setName($name) {
    $this->name = $name;
  }
  function getName() {
    return $this->name;
  }
}
```

Una vez generada la estrucura del objeto debemos crear el objeto, para ello debemos indicarlo `$banana=new Fruit();` y luego pasar los valores de ese objeto `$banana->setName('Banana');` El acceso a estas variables se puede limitar tambien a mayores de ls setter y esque podemos generar variables que solo se usen en la propia clase. Estas limitaciones se pueden aplicar tambien a las funcinoes.

- public: se puede acceder a la propiedad o método desde cualquier lugar. Esta es la opción por defecto.
- protected: se puede acceder a la propiedad o método dentro de la clase y por clases derivadas de esa clase (herencia, por ejemplo).

- private: Solo se puede acceder a la propiedad o método dentro de la clase.

$this & $instanceof

`$this` es usado cuando quieres acceder a una de las variables del propio objeto con el que estas trabajando. `$this->name=$name`. En el caso de `("x" instanceof "x")` se utiliza para comprobar si un elemento es un objeto de una clase.

Constructores y desctructores

El constructor es la función que crea de por si los objetos siguiedo las variables necesarias. Su posición dentro de la estructura suele ser debajo de la declaración de las variables.

```
  //Función de constructor al que le pasamos el nombre y lo iguala en el
objeto.
  function __construct($name) {
    $this->name = $name;
  }
```

En la posición contraria tenemos al destructo, para poder borrar el objeto que no vamos a usar más. Su posición suele ser inmediatamente posterir al constructor.

```
 function __destruct() {
  echo "The fruit is {$this->name}.";
}
```

Herencia

Llamamos herencia cuando una clase deriba de una superior (perro puede heredar de la clase animal). Esto también conlleva que las variables, funciones y métodos que se declarasen en la clase superior se pueden usar en la nueva clase. Para indicar que una clase hereda de otra se modifica el enunciado de la clase `class Dog extends Animal`. Estos métodos heredados se pueden redefinir si nombras a un método de la misma manera. DE la misma manera podemos proteger una clase para que no pueda tener clases heredadas, al nombrar una clase con `final class Animal`. Está podrá heredar de clases superiores pero no podran existir clases por debajo de ella.

Constantes de clase

Pos lo mismo que fuera de las clases, pero ahora tienen una sintaxis ligeramente diferente para declarar y para llamarlas.

```
class Goodbye {
  const LEAVING_MESSAGE = "Adiós, nos vemos pronto!";
  public function byebye() {
    echo self::LEAVING_MESSAGE;
```

```
    }
}
```

Clases abstractas

Las clases abstractas son aquellas que contienen al menos un método abstracto, no se pueden instancias de estas clases, es necesario crear una clase hija para ello.

```
abstract class ParentClass {
  abstract public function someMethod1();
  abstract public function someMethod2($name, $color);
  abstract public function someMethod3() : string;
}
```

En el caso de la clase que hereda debe de redefinir todos los métodos que aparezcan como abstractos, y estos deben tener el mismo modificador de acceso o uno menos restrictivo.

Interfaces & traits

La interfaz se puede considerar una variable de las clases abstractas. Estas no pueden tener una metodo o función concretos y solo pueden ser públicos. La utilidad de estas limitaciones es que las clases que hereden de ella no se ven restringidas a solo hereder de una interfaz. En cambio los traits son algo más sencillo, mientras que las interfaces obligan a redefinir los métodos, los traits no es necesario. A cambio tienes que definirlos en el trait. Los traits tambien permiten el uso/herencia de varios de ellos a la vez en una clase secundaria, para indicarle que utilice los traits debemos indicarlo en la primera línea antes de las variables.

```
<?php
class Persona {
    use traitA, traitB, traitC;
    // 🔹 Propiedades
    private $nombre;
    private $edad;
    // 🔹 Constructor...
```

Estáticos

Los métodos estáticos de las clases se indican en la delcaracin del propio método `public static function saludo()` se puede acceder a ellos desde cualquier parte, sin tner que crear una instancia `Saludar::saludo();`. Para poder acceder a ellos desde la propia clase debemos cambiar la sintaxis por un `self::saludo();`. Se pueden usar estos métodos desde otras clases sin tener que implementer extender o usar nada.

Objetos vacíos

Es un metodos para crear un objeto si ntener una estructura creada de antemano. Es especialmente utili para enviar información entre sistemas a través de JSON. Es una clase en particular `stdObject()`

```
$obj = new stdObject();
$obj->name = "Nick";
$obj->surname = "Doe";
$obj->age = 20;
$obj->address = "Santiago de Compostela. A Coruña";
```

Errores y excepciones

Al igual que se vio con anterioridad se pueden manejar las excepciones de dos maneras, generando nosotros la excepcion con el throw y recogiendo las que se originen con una estructura try-catch. Podemos generar las excepciones personalizadas generando una clase pripia que extienda de la clase Exception.

```
class MiExcepcion extends Exception {
    // Puedes agregar propiedades adicionales si es necesario
    private $detalle;
    // Constructor personalizado
    public function __construct($mensaje, $codigo = 0, Exception $anterior
= null, $detalle = '') {
        // Llamar al constructor de la clase base
        parent::__construct($mensaje, $codigo, $anterior);
        // Guardar el detalle adicional
        $this->detalle = $detalle;
    }
    // Método personalizado para obtener detalles
    public function obtenerDetalle() {
        return $this->detalle;
    }
}
```